

Software Engineering

Building Trinetra-AI as Real Software

PHASE 13 — RESEARCH-GRADE NOTES

Trinetra-AI

An Intelligent Multi-Sensor Fusion Framework for GPS-Denied Navigation

The algorithms of Phases 4–11 are only as good as the software that carries them. This phase is the discipline that turns a pile of notebooks into a *system*: an architecture that stays modular as it grows, version control and CI that make every result reproducible, containers that deploy identically from laptop to Jetson, tests that catch regressions, and MLOps that tracks every model and dataset. The goal is Trinetra-AI as both **research-grade** (reproducible, defensible) and **industrial-grade** (maintainable, deployable) software.

Architecture → version control & Linux → testing, Docker & structure → docs, MLOps & CI/CD → reproducible research.

Note: diagrams are drawn as native (TikZ) figures rather than Mermaid, so they render directly in the compiled PDF.

Contents

I	Foundations	3
1	Introduction to Software Engineering	3
1.1	What is software engineering?	3
1.2	Why it matters more for AI and robotics	3
1.3	Three cultures: research, industrial, scientific computing	3
1.4	The engineering lifecycle	3
1.5	The quality attributes we are engineering for	4
II	Architecture	4
2	Project Architecture	4
2.1	The architectural styles	4
III	Version Control & Environment	5
3	Git	5
3.1	Git's model	5
3.2	A branching workflow	6
3.3	Conflict resolution & examples	6
4	GitHub	7
5	Linux	7
IV	Quality, Containers & Structure	8
6	Testing	8
7	Docker	9
8	Project Structure	10
V	Docs, MLOps & Delivery	11
9	Documentation	11
10	MLOps	12

11 CI/CD	12
12 Coding Standards	13
VI Practice & Preparation	14
13 Practical Implementation	14
13.1 The complete Trinetra-AI software-engineering workflow	14
13.2 Development, testing, and deployment workflows	15
14 Interview & Research Preparation	15
14.1 60 interview questions	15
14.2 40 viva questions	17
14.3 30 coding questions	18
15 Summary, Glossary & Roadmap	18
15.1 Summary	18
15.2 Glossary	19
15.3 Roadmap: Software Engineering → Research Knowledge	19

Part I — Foundations

1 Introduction to Software Engineering

1.1 What is software engineering?

Software engineering is the disciplined application of engineering principles to the *whole life* of software — design, construction, testing, documentation, deployment, and maintenance — so that systems are correct, maintainable, and evolvable by a team over time. It is the difference between *code that runs once* and *software that lives*.

Intuition: Phases 4–11 produced *algorithms*; software engineering produces a *product*. A brilliant fusion filter buried in an unversioned, untested, undocumented notebook is a liability — nobody (including future-you) can reproduce, extend, or trust it. This phase makes the work durable.

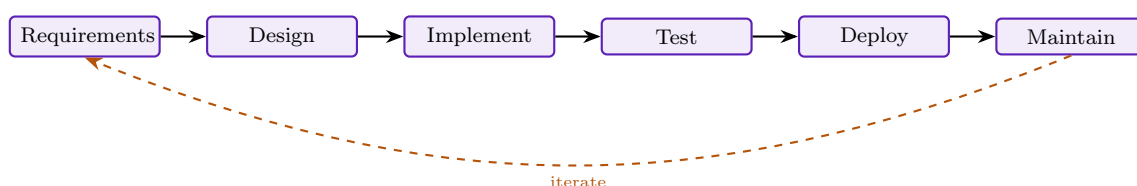
1.2 Why it matters more for AI and robotics

Trait	Why Trinetra-AI feels it acutely
AI adds <i>data & models</i> as first-class artifacts	code alone doesn't define behaviour; the dataset and trained weights do too (needs data/model versioning)
Silent failure	a mis-scaled sensor or stale model degrades accuracy without crashing; only tests/monitoring catch it
Real-time & embedded	the code must run deterministically on a Jetson, not just pass on a laptop
Multi-language	Python (AI) + C++ (real-time ROS 2 nodes) must coexist cleanly
Reproducibility is the deliverable	a research result you cannot reproduce is not a result

1.3 Three cultures: research, industrial, scientific computing

Research software engineering (RSE) optimises for correctness, reproducibility, and rapid iteration on ideas. **Industrial SE** optimises for maintainability, reliability, and scale over a long lifetime with many contributors. **Scientific computing** adds numerical correctness and performance. Trinetra-AI lives at their intersection: it must be *reproducible like research*, *deployable like industry*, and *numerically sound like scientific computing*.

1.4 The engineering lifecycle



For a research project this loop is *agile and incremental*: short cycles, each ending in working, tested, version-tagged software, not a single big-bang delivery.

1.5 The quality attributes we are engineering for

Attribute	Meaning	Trinetra-AI lever
Maintainability	easy to understand & change	modular architecture, clean code, docs
Scalability	handles more sensors/data/nodes	modular ROS 2 nodes, data pipeline
Reliability	works correctly under real conditions	tests, CI, monitoring, error handling
Reproducibility	same inputs → same results	Git + DVC + MLflow, containers, seeds
Portability	runs anywhere identically	Docker, pinned dependencies

These five attributes are the *acceptance criteria* for Trinetra-AI as software. Reproducibility is first among equals: it is what converts “my model got 2.1 m error” into a defensible thesis claim. Every practice in this phase (version control, containers, testing, MLOps) ultimately serves it. Key references: Wilson et al., *Best Practices for Scientific Computing* (2014) and *Good Enough Practices* (2017); Sandve et al., *Ten Simple Rules for Reproducible Computational Research* (2013).

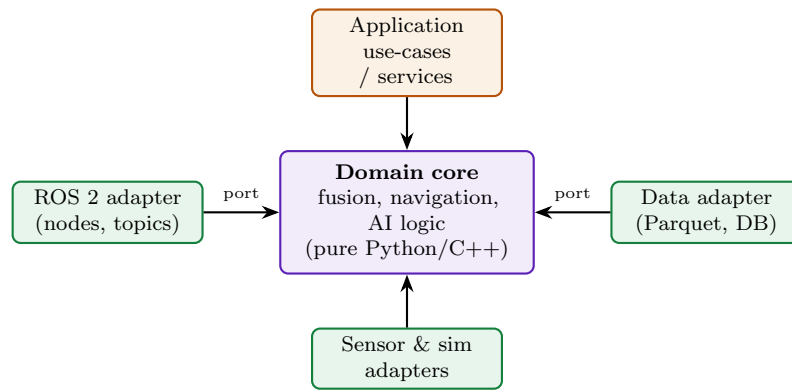
Part II — Architecture

2 Project Architecture

2.1 The architectural styles

Style	Idea	Fit for Trinetra-AI
Monolithic	one deployable unit	simple but rigid; poor for many sensors/modules
Modular	one codebase, clear internal modules	yes at code level
Layered	horizontal layers (data → domain → app)	yes for the core library
Microservices	independent networked services	overkill for a single vehicle
Plugin	core + swappable extensions	yes for sensors & AI modules
Clean / Hexagonal	domain isolated behind ports/adapters	yes to keep the algorithm core pure

Clean (Hexagonal / Ports-and-Adapters) architecture puts the pure domain logic (the fusion & navigation algorithms) at the centre, depending on *nothing* external. Everything volatile (ROS 2, sensor drivers, databases, UI) sits at the edges as **adapters** plugged into **ports** (interfaces). Dependencies point *inward*: the core never imports ROS. This lets you test the algorithms without a robot and swap ROS 2 for a bag replay or a unit test.



dependencies point inward; the core imports nothing external

Verdict for Trinetra-AI: a **modular, layered, clean/hexagonal** core wrapped by a **plugin** pattern for sensors and AI modules, exposed through ROS 2 adapters. Concretely: a pure `trinetra_core` library (no ROS import) holding the estimators/algorithms; thin ROS 2 nodes (Phase 10) as adapters; sensors and AI models registered as plugins behind interfaces. *Not* microservices (a single vehicle doesn't need network-partitioned services; the complexity cost isn't justified). This keeps the science testable in isolation and the system swappable at the edges.

Part III — Version Control & Environment

3 Git

3.1 Git's model

Git is a distributed version-control system: every clone holds the full history. A **repository** is a project's history as a directed acyclic graph of **commits** (immutable snapshots identified by a SHA-1 hash). A **branch** is a movable pointer to a commit; **HEAD** points to your current branch. Git stores *snapshots*, not *diffs*, and is content-addressed, which is why branching and merging are cheap.

Operation	What it does	When (Trinetra-AI)
commit	snapshot staged changes	every logical unit of work
branch	diverge a line of development	per feature / experiment
merge	join branches (creates a merge commit)	integrate a finished feature
rebase	replay commits onto a new base	tidy a feature branch before merge
cherry-pick	copy one commit elsewhere	hotfix onto a release branch
tag	name a commit (e.g. <code>v1.0</code>)	mark thesis milestones / releases
remote	a linked copy (e.g. GitHub)	backup & collaboration

4 GitHub

GitHub is a hosting platform that adds collaboration on top of Git: a shared **remote** plus **pull requests** (propose & review changes), **issues** (track bugs/tasks), **Projects** (boards), **Releases** (versioned artifacts), **Actions** (CI/CD), and **branch protection** (enforce review & passing tests before merge). A **fork** is a personal copy of someone else's repo; the open-source loop is fork → branch → PR → review → merge.

Feature	Purpose	Trinetra-AI use
Pull Request	review + CI gate before merge	every feature enters main via PR
Issues	track bugs, tasks, ideas	backlog per phase/module
Projects	kanban board	plan the thesis milestones
Releases	tagged, downloadable versions	snapshot code for each paper/demo
Actions	automated CI/CD	test + build + docker on every push
Secrets	encrypted credentials for CI	registry tokens, deploy keys
Branch protection	require review + green CI	keep main always working

Even as a solo researcher, treat Trinetra-AI like an open-source project: protect **main** (no direct pushes; PRs must pass CI), file issues for every task so the history documents your reasoning, and cut a **Release** for each thesis milestone. This discipline is what makes the repo credible if you open-source it as a contribution, and it forces the reproducibility that the thesis needs. A good **README**, **LICENSE**, and **CITATION.cff** belong at the repo root.

5 Linux

Linux is the operating system of robotics: ROS 2, the Jetson, and virtually all robotics tooling target it. Fluency in the **terminal**, **filesystem**, **permissions**, **processes**, **networking/SSH**, **bash** scripting, **systemd** services, and **package managers** is a prerequisite for developing and deploying Trinetra-AI.

Area	Key idea	Trinetra-AI relevance
Filesystem & paths	everything is a file; tree from /	workspace layout, device nodes
Permissions	user/group/other, rxw	access to /dev sensor devices
Users & groups	least privilege	add user to dialout for serial IMU
Processes	ps , top , signals	monitor node CPU on the Jetson
Networking & SSH	remote access, key auth	headless deploy to the robot
Bash	glue scripting	bring-up & data-collection scripts
systemd	manage services at boot	auto-start the stack on the robot
Package managers	apt , pip , rosdep	reproducible dependency install

```
# Essential commands you will use constantly on the robot
ls -lah cd pwd cp mv rm mkdir # navigation & files
```



```

chmod +x run.sh chown user:user file # permissions
sudo usermod -aG dialout $USER # allow serial IMU access
ps aux | grep ros top htop # processes & load
ip addr ping robot.local ssh user@robot # networking & remote
journalctl -u trinetra.service -f # follow a systemd service log
df -h du -sh * free -h # disk & memory
tar czf logs.tgz logs/ scp logs.tgz user@host: # archive & copy off-robot

```

```

# /etc/systemd/system/trinetra.service -- auto-start the stack at boot
[Unit]
Description=Trinetra-AI ROS 2 stack
After=network.target
[Service]
ExecStart=/opt/trinetra/scripts/bringup.sh
Restart=on-failure
User=trinetr
[Install]
WantedBy=multi-user.target

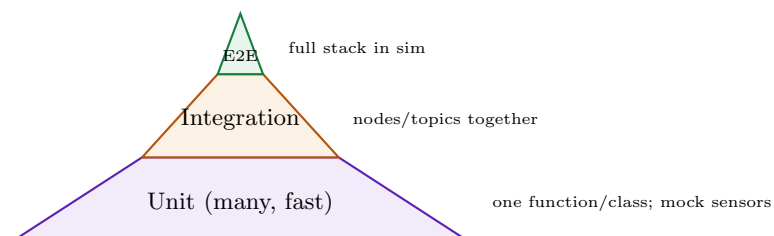
```

On the Jetson, Trinetra-AI runs headless: you **ssh** in, a **systemd** unit auto-starts the ROS 2 stack at boot and restarts it on failure, and **journalctl** streams logs. A handful of **bash** scripts (**bringup.sh**, **record.sh**, **deploy.sh**) wrap the common workflows. Getting sensor access right is often a *permissions* problem (adding your user to **dialout**/**video** for serial and camera devices), so understanding Linux permissions is not optional.

Part IV — Quality, Containers & Structure

6 Testing

Testing is executable evidence that the software does what it should. The **test pyramid**: many fast **unit** tests (one function/class), fewer **integration** tests (modules together), and a few slow **system/end-to-end** tests (the whole stack). **Regression** tests lock in fixed bugs; **acceptance** tests check user-level requirements.



Level	Scope	Trinetra-AI example
Unit (pytest)	one function, isolated	EKF predict step math; unit conversions
Mocking	fake dependencies	feed a mock IMU stream to a node
Integration	modules together	sync + fusion nodes on a recorded bag
System / E2E	whole stack in sim	Gazebo GPS-denied run → trajectory error bound
Regression	re-run a fixed bug	a known drift case stays fixed
AI model testing	behavioural/metric checks	error < threshold on a held-out trajectory; no NaNs; invariances

```
# pytest: unit-test the EKF update with a mock, and a metric-based model test
import numpy as np, pytest
from trinetra_core.fusion import EKF

def test_ekf_update_reduces_covariance():
    ekf = EKF(dim=2); P0 = np.trace(ekf.P)
    ekf.update(z=np.array([1.0, 0.0]), R=np.eye(2)*0.1)
    assert np.trace(ekf.P) < P0 # a measurement must not increase uncertainty

@pytest.mark.parametrize("traj", ["straight", "loop", "figure8"])
def test_model_error_within_budget(traj, load_bag):
    est, gt = run_pipeline(load_bag(traj))
    ate = np.sqrt(((est - gt) ** 2).sum(1).mean())
    assert ate < 3.0 # metric gate: ATE under 3 m
    assert not np.isnan(est).any() # no NaNs leak through
```

Trinetra-AI's testing has an *AI-specific* twist: models can't be asserted equal to a golden value, so tests are **metric gates** (ATE below a budget), **invariance checks** (rotate the input, the predicted trajectory rotates), and **sanity checks** (no NaNs, covariance stays positive-definite). Combine `pytest` for the pure core with ROS 2 `launch_testing` for node integration, and measure **coverage** to find untested paths. Ship nothing that a red test touches.

7 Docker

Docker packages software with its dependencies into an **image** (a read-only template) that runs as a **container** (an isolated process). **Volumes** persist data outside the container; **networks** connect containers; a **Dockerfile** is the recipe; **Docker Compose** orchestrates multiple containers. It solves “works on my machine” by making the environment itself version-controlled and portable.

Concept	What it is	Trinetra-AI use
Image / container	template / running instance	reproducible ROS 2 environment
Dockerfile	build recipe	pin ROS distro + Python deps
Multi-stage build	build stage → slim runtime	small image for the Jetson
Volume	persistent host storage	mount datasets & model weights
Network	inter-container comms	DDS between sensor/fusion containers
GPU container	CUDA passthrough	TensorRT inference on Jetson
Compose	multi-container orchestration	bring up the whole stack

```
# Dockerfile -- multi-stage: heavy build, slim runtime (ASCII only)
FROM ros:humble AS build
WORKDIR /ws
COPY . src/trinetra
RUN . /opt/ros/humble/setup.sh && colcon build --symlink-install

FROM ros:humble-ros-core AS runtime # slimmer base for deployment
COPY --from=build /ws/install /ws/install
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
CMD ["bash", "-lc", ". /ws/install/setup.sh && ros2 launch trinetra bringup.launch.py"]
```

```
# docker-compose.yaml -- dev + inference services sharing the host network for DDS
services:
  trinetra:
    build: .
    network_mode: host # ROS 2 DDS discovery on host net
    volumes:
      - ./data:/ws/data # datasets persist on host
      - ./models:/ws/models
    deploy:
      resources:
        reservations:
          devices: [{"capabilities": ["gpu"]} # GPU passthrough for TensorRT
```

Trinetra-AI containers: one **Dockerfile** pins the exact ROS 2 distro and Python dependencies so the environment is reproducible byte-for-byte; a **multi-stage** build keeps the deployed Jetson image small; **volumes** mount datasets/weights so they are not baked into the image; **GPU passthrough** enables TensorRT inference; and **Compose** brings the stack up with one command on both laptop and robot. The container *is* the reproducibility guarantee for “it ran and produced these numbers.”

8 Project Structure

The repository layout encodes the architecture (Sec 2): a pure core, thin ROS 2 adapters, plugins for sensors/AI, and clear homes for data, configs, tests, docs, and research.

```
trinetra-ai/
|- src/
| |- trinetra_core/ # PURE domain logic (no ROS import) -- the science
| | |- sensor_fusion/ # EKF/UKF/PF estimators (Phase 7)
| | |- navigation/ # INS, dead-reckoning, map-matching (Phase 8)
| | |- ai_modules/ # denoise, drift, anomaly, confidence (Phase 9)
| | +- pipelines/ # sync, clean, feature build (Phase 11)
| +- ros2/ # ROS 2 adapters: nodes, launch, msgs (Phase 10)
```

```

|- configs/ # YAML: ekf.yaml, qos.yaml, sensor calibrations (Hydra)
|- datasets/ # DVC-tracked data (pointers in git, blobs in remote)
|- models/ # DVC-tracked weights + ONNX/TensorRT exports
|- visualization/ # dashboards, RViz2 configs, plotting (Phase 12)
|- tests/ # unit / integration / system (pytest, launch_testing)
|- docs/ # Sphinx/MkDocs site, architecture & design docs
|- research/ # papers, notes (Phases 4-13), experiment logs
|- docker/ # Dockerfile(s), docker-compose.yaml
|- scripts/ # bringup.sh, record.sh, deploy.sh, train.sh
|- .github/workflows/ # CI/CD: test, build, docker, docs
|- pyproject.toml # deps, tooling (black, ruff, pytest) config
|- dvc.yaml .dvc/ # data/ML pipeline stages
|- README.md LICENSE CITATION.cff

```

The load-bearing decision is the **trinetra_core** vs **ros2** split: the **core imports no ROS**, so the algorithms are unit-testable and reusable without a robot, while **ros2/** holds only thin adapters. **datasets/** and **models/** are **DVC-tracked** (never committed to Git); **research/** keeps the phase notes and experiment logs beside the code; **configs/** centralises every tunable so nothing is hard-coded. This structure *is* the clean architecture made concrete on disk.

Part V — Docs, MLOps & Delivery

9 Documentation

Documentation makes software usable and reproducible by others (and future-you). Layers: a **README** (what/why/how-to-run), **API docs** (generated from docstrings), **architecture & design docs** (the “why” of structure), **code comments** (non-obvious intent), and **research documentation** (methods, experiments, results for the thesis).

Tool	Produces	Trinetra-AI use
Sphinx	Python API docs from docstrings	trinetra_core reference
MkDocs (Material)	Markdown docs site	user guide, architecture, tutorials
Doxygen	C++ API docs	the C++ ROS 2 nodes
docstrings (NumPy/Google)	inline API contract	every public function
ADRs	architecture decision records	why clean architecture, why Humble

Trinetra-AI documents at three altitudes: **Sphinx** (Python) + **Doxygen** (C++) for auto-generated API reference; **MkDocs** for the human-facing guide and architecture; and the **research/** notes (Phases 4–13) as the scientific record. Good docstrings are not optional here: they are the API contract the tests check and the reference the thesis cites. Write the README so a stranger can reproduce your headline result in under ten minutes.

10 MLOps

MLOps extends software engineering to the extra artifacts of AI: **data** and **models**. It adds **experiment tracking** (params/metrics/artifacts per run), **data & model versioning**, a **model registry** (named, staged model versions), **deployment**, and **monitoring** (detecting drift/degradation in production).

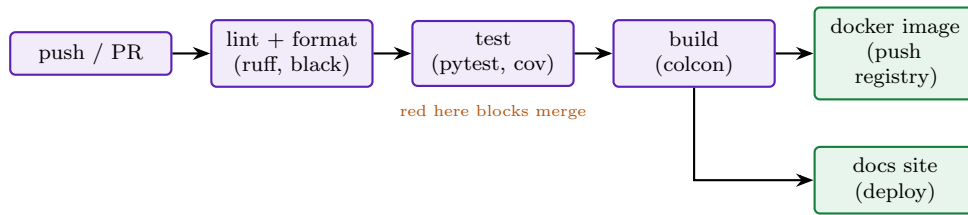
Tool	Role	Trinetra-AI use
Git	code versioning	the whole codebase
DVC	data & model versioning (Git-linked)	sensor corpus, weights, ONNX exports
MLflow	experiment tracking + registry	every training run's params/metrics/model
Hydra	config management	reproducible run configurations
Monitoring	drift/health in deployment	anomaly & confidence signals (Phase 9)

```
# Track a training run so it is fully reproducible (code SHA + data ver + metrics)
import mlflow, subprocess
sha = subprocess.check_output(["git", "rev-parse", "--short", "HEAD"]).decode().strip()
with mlflow.start_run(run_name=f"drift-tcn-{sha}"):
    mlflow.log_params({"model": "TCN", "window": 200, "lr": 1e-3, "git_sha": sha,
                      "data_version": "v3"}) # ties run to code + data
    model = train(...)
    mlflow.log_metric("val_ate", ate)
    mlflow.pytorch.log_model(model, "model") # registry-ready artifact
```

The reproducibility triangle for Trinetra-AI is **Git (code) + DVC (data/models) + MLflow (runs)**: each experiment records the code SHA, the dataset version, the config, and the metrics, so “model X scored 2.1m ATE on dataset v3” is fully reconstructable months later. This is the single most important engineering investment for a *research-grade* project; it is what makes the thesis's numbers defensible. Key references: Sculley et al., *Hidden Technical Debt in ML Systems* (NeurIPS 2015); Amershi et al., *Software Engineering for Machine Learning* (ICSE-SEIP 2019).

11 CI/CD

Continuous Integration (CI) runs tests automatically on every push, catching breakage early; **Continuous Delivery/Deployment (CD)** automatically builds and ships artifacts (Docker images, docs) when tests pass. On GitHub this is **GitHub Actions**: workflows of jobs triggered by events, defined in `.github/workflows/`.



```

# .github/workflows/ci.yaml -- test + build on every push/PR (ASCII only)
name: ci
on: [push, pull_request]
jobs:
  test:
    runs-on: ubuntu-latest
    container: ros:humble
    steps:
      - uses: actions/checkout@v4
      - run: pip install -r requirements.txt
      - run: pip install pytest pytest-cov ruff
      - run: ruff check src/ # lint gate
      - run: pytest --cov=trinetra_core tests/ # test + coverage gate
  docker:
    needs: test # only if tests pass
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - run: docker build -t trinetra:${{ github.sha }} .

```

Trinetra-AI runs five logical pipelines on GitHub Actions: **test** (pytest + coverage gate), **lint/format** (ruff + black), **build** (colcon), **docker** (build & push the image), and **docs** (publish the site). Branch protection makes a green **test** pipeline mandatory before merge, so **main** is always deployable. A *research pipeline* can additionally re-run a small benchmark on a fixed dataset and fail if accuracy regresses beyond a threshold, guarding the thesis's core result automatically.

12 Coding Standards

Coding standards keep a codebase consistent and readable so a team (and future-you) can work in it. **PEP 8** is Python's style guide (enforced by **black/ruff**). **DRY** (don't repeat yourself), **KISS** (keep it simple), and **YAGNI** (you aren't gonna need it) are complexity-control heuristics. **Clean Code** (Martin) is the craft of readable, intention-revealing code.

SOLID guides maintainable object-oriented design. **S**ingle responsibility (one reason to change); **O**pen/closed (extend without modifying); **L**iskov substitution (subtypes honour their base contract); **I**nterface segregation (small, focused interfaces); **D**ependency inversion (depend on abstractions, not concretions). The last one is exactly what makes the clean architecture of Sec 2 possible: the core depends on a **SensorSource** interface, not on ROS.

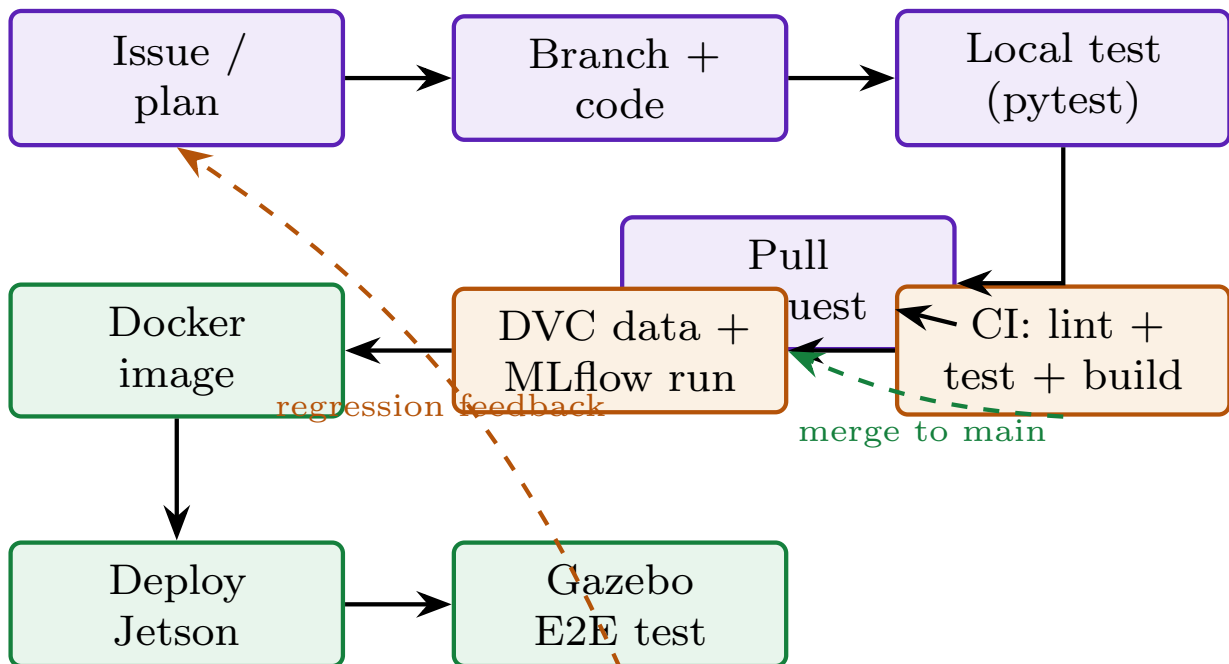
Principle/pattern	Essence	Trinetra-AI application
PEP 8 / black / ruff	consistent style, auto-formatted	enforced in CI
DRY	one source of truth	one feature function (train==serve)
KISS / YAGNI	no speculative complexity	no microservices, no premature abstraction
Strategy pattern	swappable algorithms	pick EKF vs UKF vs learned filter
Factory pattern	construct by config	build a sensor/AI plugin from YAML
Adapter pattern	wrap an external API	ROS 2 nodes wrapping the pure core
Observer (pub/sub)	decoupled events	ROS 2 topics (Phase 10)

The design patterns that recur in Trinetra-AI are **Strategy** (swap estimators/AI models behind a common interface), **Factory** (build sensor/AI plugins from config), and **Adapter** (ROS 2 wrapping the pure core) — the pub/sub **Observer** pattern is already provided by ROS 2 topics. Applied with **SOLID** and enforced by **ruff/ black** in CI, these keep the system extensible (add a sensor or model without editing the core) and readable (a reviewer can follow it). References: Martin, *Clean Code* (2008); Gamma et al., *Design Patterns* (1994); Fowler, *Refactoring* (2018).

Part VI — Practice & Preparation

13 Practical Implementation

13.1 The complete Trinetra-AI software-engineering workflow



13.2 Development, testing, and deployment workflows

Workflow	Steps
Development	issue → short-lived branch → code + docstrings → local <code>pytest/ruff</code> → PR → review + green CI → merge to <code>main</code> → tag milestone
Testing	unit (<code>pytest</code>) → integration (<code>launch_testing</code> on bags) → system (Gazebo E2E, ATE budget) → regression on fixed cases; coverage tracked
Data / model	version data & weights with DVC → train with config (Hydra) → log run to MLflow (code SHA + data ver) → export ONNX/TensorRT
Deployment	CI builds Docker image → push to registry → pull on Jetson → <code>docker compose up</code> / <code>systemd</code> auto-start → monitor via logs & health topics

Best practices. Pure core with no ROS import; one feature/experiment per branch; protect `main` behind CI; version code+data+models together; pin every dependency; containerise the environment; write docstrings and a reproducible README; tag every thesis result with its commit SHA. **Common mistakes.** Committing data/weights to Git; train/serve config drift; untested “glue” scripts that silently break; a core that imports ROS (untestable offline); unpinned dependencies (“worked last month”); no seed control (irreproducible runs); over-engineering (microservices for one robot).

14 Interview & Research Preparation

14.1 60 interview questions

1. Define software engineering.
2. Why does SE matter more for AI?
3. How is robotics software different?
4. RSE vs industrial SE vs scientific computing.
5. Name the quality attributes.
6. What is maintainability?
7. What is reproducibility and why does it matter?
8. The software lifecycle stages.
9. Agile vs waterfall.
10. Monolithic vs modular.
11. What is layered architecture?
12. Microservices trade-offs.
13. Plugin architecture.
14. Clean/hexagonal architecture.
15. Ports and adapters.
16. Why not microservices for Trinetra-AI?
17. What is Git?
18. Snapshots vs diffs.
19. What is a commit?
20. Branch vs tag.
21. Merge vs rebase.

22. When to rebase.
23. What is cherry-pick?
24. What is a remote?
25. Resolve a merge conflict.
26. Trunk-based development.
27. Conventional Commits.
28. What is GitHub?
29. Fork vs clone.
30. What is a pull request?
31. Branch protection.
32. What are GitHub Actions?
33. What are secrets?
34. Open-source workflow.
35. Why Linux for robotics?
36. File permissions (rwx).
37. What is systemd?
38. apt vs pip vs rosdep.
39. What is SSH?
40. What is a bash script for?
41. The test pyramid.
42. Unit vs integration vs E2E.
43. What is mocking?
44. What is coverage?
45. How do you test an AI model?
46. Metric-gate tests.
47. What is Docker?
48. Image vs container.
49. What is a volume?
50. Multi-stage build.
51. GPU containers.
52. What is Docker Compose?
53. What is MLOps?
54. Git vs DVC.
55. What does MLflow track?
56. What is a model registry?
57. What is CI/CD?
58. What is PEP 8?
59. Explain SOLID.
60. DRY vs KISS vs YAGNI.

14.2 40 viva questions

1. Justify the architecture you chose for Trinetra-AI.
2. Why must the core not import ROS?
3. Draw the ports-and-adapters diagram.
4. How does dependency inversion enable testing?
5. Design the repo folder structure.
6. Where do data and models live, and why not Git?
7. Design a Git branching workflow for experiments.
8. How do you tie a result to its exact code + data?
9. Explain your CI pipeline stages.
10. What makes `main` always deployable?
11. Design a metric-gate test for the fusion output.
12. How do you test a ROS 2 node?
13. How do you test without hardware?
14. Explain your Docker multi-stage build.
15. How does the container guarantee reproducibility?
16. Deploy the stack to a Jetson.
17. Set up systemd auto-start.
18. Explain the reproducibility triangle.
19. Log a reproducible MLflow run.
20. Version a dataset with DVC.
21. Detect model drift in deployment.
22. Apply SOLID to the estimator classes.
23. Where do Strategy/Factory/Adapter appear?
24. Why avoid microservices here?
25. Handle train/serve config drift.
26. Your documentation strategy (Sphinx/MkDocs/Doxygen).
27. What goes in the README.
28. Write an ADR for choosing Humble.
29. Enforce style in CI.
30. Measure and act on coverage.
31. Guard the thesis result with a research pipeline.
32. Roll back a bad deployment.
33. Manage secrets in CI.
34. Pin dependencies reproducibly.
35. Handle a merge conflict in `ekf.py`.
36. Structure integration tests on bags.
37. Seed control for reproducibility.
38. Scale from one robot to many.
39. Biggest SE risk in the project.
40. How SE makes the research defensible.

14.3 30 coding questions

1. Write a `pytest` for the EKF update.
2. Parametrize a test over trajectories.
3. Mock an IMU stream for a node test.
4. Write a metric-gate ($ATE < budget$) test.
5. Assert covariance stays positive-definite.
6. A ROS 2 `launch_testing` case.
7. Measure coverage and fail under 80%.
8. Define a `SensorSource` interface (port).
9. Implement a ROS 2 adapter for it.
10. Strategy pattern for EKF/UKF selection.
11. Factory building a plugin from YAML.
12. A Git pre-commit hook running ruff.
13. Script to tag a commit with an experiment id.
14. Log params/metrics/model to MLflow.
15. DVC-track a dataset and push.
16. Write a multi-stage Dockerfile.
17. A `docker-compose.yaml` with GPU + volumes.
18. A GitHub Actions test workflow.
19. Add a docker-build job gated on tests.
20. A systemd unit for the stack.
21. A bash `bringup.sh`.
22. Grant serial-device permission (udev/group).
23. A Sphinx-ready NumPy docstring.
24. Config with Hydra + override.
25. Seed all RNGs for reproducibility.
26. A regression test for a fixed drift bug.
27. Pin deps in `pyproject.toml`.
28. Pre-commit config (black+ruff).
29. Detect drift from a metrics stream.
30. Semantic-version & tag a release.

15 Summary, Glossary & Roadmap

15.1 Summary

Software engineering is what turns Trinetra-AI from a set of algorithms into a *reproducible, deployable system*. The architecture is a **modular, clean/hexagonal core** (pure fusion/navigation/AI logic that imports nothing external) wrapped by **ROS 2 adapters** and **sensor/AI plugins** — not microservices. **Git** (trunk-based, Conventional Commits, tagged milestones) and **GitHub** (PRs, branch protection, Actions) govern change; **Linux** skills (permissions, systemd, bash, SSH)

run it on the Jetson; **tests** (unit → integration → E2E, with AI metric-gates) protect correctness; **Docker** (multi-stage, GPU, Compose) makes the environment portable; **MLOps** (Git + DVC + MLflow) versions code, data, and models together; **CI/CD** automates it all; and **coding standards** (PEP 8, SOLID, DRY/KISS/YAGNI, patterns) keep it clean. The through-line is **reproducibility**: the engineering that makes every thesis number traceable and defensible.

15.2 Glossary

Adapter — code wrapping an external API to a core interface (ROS 2 node). **Clean/Hexagonal** — architecture isolating pure domain logic behind ports. **CI/CD** — automated test/build/deploy on every change. **Conventional Commits** — structured commit messages (`feat:`, `fix:`). **Coverage** — fraction of code exercised by tests. **DVC** — Git-linked data/model versioning. **MLflow** — experiment tracking + model registry. **MLOps** — SE practices for data & models. **Port** — an interface the core depends on (not a concrete tool). **Rebase** — replay commits onto a new base for a linear history. **SOLID** — five OO design principles. **systemd** — Linux service manager (auto-start at boot). **Test pyramid** — many unit, fewer integration, few E2E tests.

15.3 Roadmap: Software Engineering → Research Knowledge

Enables	How SE serves the research
Reproducible results	Git + DVC + MLflow make every figure traceable to code + data tests & CI guarantee the numbers, containers guarantee the environment
Defensible thesis	
Extensible platform	clean architecture + plugins let new sensors/models slot in
Credible open-source	PRs, docs, releases make Trinetra-AI a usable contribution
Research knowledge (next)	a disciplined codebase is the foundation for writing papers, running rigorous experiments, and defending the work

Why research knowledge comes next. With the software engineered — architected, versioned, tested, containerised, and tracked — the remaining work is *scholarly*: framing the contribution against the literature (Phase’s literature database), designing rigorous experiments, reporting results honestly, and writing the thesis and IEEE papers. Sound software engineering is the precondition for sound research: it is what lets you claim a result and *prove* it. Trinetra-AI is now not just an idea, but a system you can stand behind.

End of Phase 13 notes. Trinetra-AI is now engineered as reproducible, deployable, research-grade software: architecture, version control, testing, containers, MLOps, and CI/CD in service of defensible science.